The pline Package,
Version 0.01

https://github.com/Pseudonym321/TikZ-Animations/tree/master1/TikZ/pline

Jasper

July 8, 2025

Contents

Chapter 1

Introduction

pline is a Lua-based 3D graphics package for Ti*k*Z.

1.1 What are the current capabilities of pline?

pline can render multiple non-intersecting triangulated mesh surfaces at once. It is also possible to draw a plane which has been clipped by a cube. There are commands for drawing and labelling geometric vectors too. Curves and surfaces can be defined using matrix transformations.

1.2 How does pline improve on existing packages?

There are two significant packages for 3D artwork in Ti*k*Z: pgfplots and tikz-3dplot. pgfplots can z-sort the faces of a single parametric surface, but not multiple surfaces at once. Both pgfplots and tikz-3dplot are capable of changing the camera view, but the result is necessarily orthographic, and the rotations are restricted to azimuth and elevation; that is, the z-axis is always vertical. Neither pgfplots nor tikz-3dplot enable matrix pline is built to render multiple parametric objects at once, enable arbitrary object orientations in non-orthographic projections, and enable matrix

1.3 Where is the package at, and where it is heading?

The vision of the package pline is to one day be able to render arbitrary numbers of intersecting polygons, by clipping them with each other; this would enable arbitrary surface intersections, without visual artefacts. A more near term goal of pline is to also implement an ODE solving tool which approximates trajectories in vector fields. I also want to add capabilities for doing programming in Lua, instead of in pgf

Chapter 2

Geometric Vectors

2.1 How are geometric vectors defined in pline?

Geometric vectors are defined by two things: a start-coordinate, and a relative displacement. There are two classifications of geometric vectors; there are zero vectors, and non-zero vectors. A zero vector is represented by a small circle. A non-zero vector is represented by a line segment with an arrow. The endpoints of a non-zerovector may or may not have a circle centered on the endpoint. If there is no circle drawn, the arrow stops directly at the endpoint. If a circle is drawn, then the arrow tip is moved back by the circle's radius. Observe the tips of the vectors in Figure 2.1.1. Both tips lie on the same point, despite one arrow tip being offset by the circle's radius. The syntax for drawing Figure 2.1.1 is shown in the following code.

```
\begin{tikzpicture}
    \drawvector[points = both,yellow]{0,0}{2,0}
    \drawvector[blue]{4,0}{-2,0}
\end{tikzpicture}
```

There are four values for the key `points`: `neither`, `behind`, `front` and `both`. These tell the command `\drawvector` whether and where to put circles over the endpoints. See Figure 2.1.2, which uses the following code.

```
% a)
\begin{tikzpicture}
    \drawvector[points = neither]{0,0}{0,1}
\end{tikzpicture}
% b)
\begin{tikzpicture}
    \drawvector[points = behind]{1,0}{0,1}
\end{tikzpicture}
% c)
```


Figure 2.1.1:
The exact location of a geometric vector's tip.

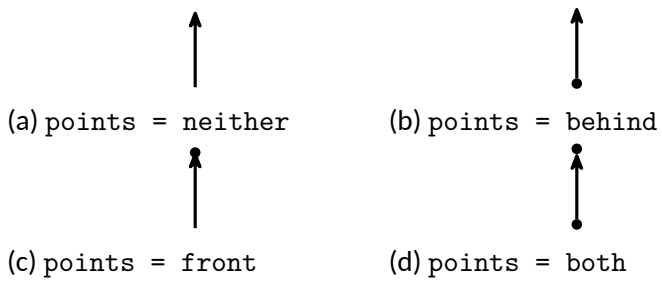

Figure 2.1.2:
The values for the key `points`.

```
\begin{tikzpicture}
    \drawvector[points = front]{2,0}{0,1}
\end{tikzpicture}
% d)
\begin{tikzpicture}
    \drawvector[points = both]{3,0}{0,1}
\end{tikzpicture}
```

Chapter 3

Parametric Objects

3.1 How is a parametric object defined?

Parametric objects are defined using a handful of different pieces of information. In particular, they use parametric functions, domain parameters and a samples parameter for each dimension in the domain. Figure 3.1.1 is defined by the following code:

```
\begin{tikzpicture}
    \appendcurve[
        u min = 0
        ,u max = {tau}
        ,u samples = 200
        ,transformation = {euler(pi/2,pi/3,pi/2)}
        ,x = {2*cos(u)}
        ,y = {2*sin(u)}
        ,z = {u}
        ,draw options = {
            purple
            ,ultra thick
            ,line join = round
        }
    ]
    \appendsurface[
        u min = 0
        ,u max = {tau}
        ,u samples = 36
        ,v min = 0
        ,v max = {tau}
        ,v samples = 36*2/3
        ,transformation = {euler(pi/2,pi/3,pi/2)}
        ,x = {2*cos(u)+sphere(u,v)[1][1]}
        ,y = {2*sin(u)+sphere(u,v)[1][2]}
        ,z = {u+sphere(u,v)[1][3]}
```

```
        ,draw options = {
            draw
            ,blue,ultra thin
            ,line join = round
        }
        ,fill options = {
            fill = green
            ,fill opacity = 0.7
        }
    ]
    \foreach \u in {0,...,35} {
        \foreach \v in {0,...,35} {
            \pgfmathsetmacro{\uscale}{2*pi/(36-1)}
            \pgfmathsetmacro{\vscale}{2*pi/(36*2/3-1)}
            \pgfmathsetmacro{\u}{\u*\uscale}
            \pgfmathsetmacro{\v}{\v*\vscale}
            \appendcurve[
                u min = 0
                ,u max = 0.3
                ,u samples = 2
                ,transformation = {euler(pi/2,pi/3,pi/2)}
                ,x = {
                    2 * cos(token.get_macro("u")) +
                    (u + 1) * sphere(token.get_macro("u"),token.get_macro("v"))[1][1]
                }
                ,y = {
                    2 * sin(token.get_macro("u")) +
                    (u + 1) * sphere(token.get_macro("u"),token.get_macro("v"))[1][2]
                }
                ,z = {
                    token.get_macro("u") +
                    (u + 1) * sphere(
                        token.get_macro("u"),
                        token.get_macro("v")
                    )[1][3]
                }
                ,draw options = {-{Stealth[round]}}
            ]
        }
    }
    \rendersegments
\end{tikzpicture}
```

And the code for Figure 3.1.2 is as follows.

```
\begin{tikzpicture}
```

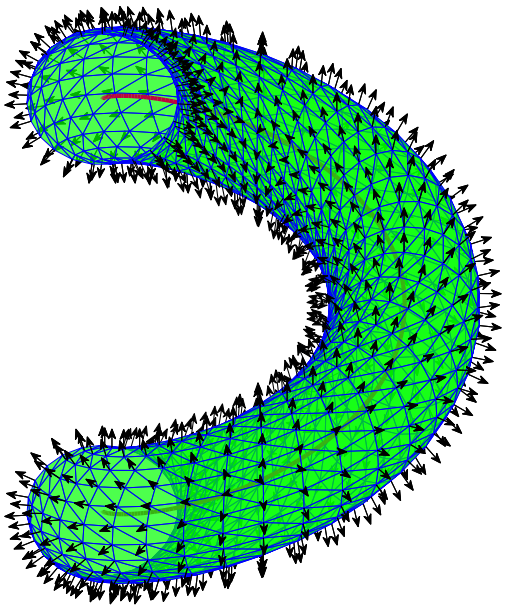

Figure 3.1.1:
A triangulated mesh surface and a curve.

```
\clip (-5,-5) rectangle (5,5);
\pgfmathsetmacro{\angle}{0}
\let\angle\angle
\foreach \longitude in {1,...,36}{
    \let\longitude\longitude
    \appendcurve[
        u min = 0
        ,u max = {pi}
        ,u samples = 4*45
        ,transformation = {euler(pi/2,pi/3,pi/6)}
        ,x = {
            stereographic_projection(
                matrix_multiply(
                    sphere(2*pi*token.get_macro("longitude")/36,u)
                    ,euler(0,pi/2,(2*pi/24)*token.get_macro("angle")/36)
                )
            )[1][1]
        }
        ,y = {
            stereographic_projection(
                matrix_multiply(
                    sphere(2*pi*token.get_macro("longitude")/36,u)
                    ,euler(0,pi/2,(2*pi/24)*token.get_macro("angle")/36)
                )
            )[1][2]
```

```
        }
        ,z = 0*u
    ]
    \appendcurve[
        u min = 0
        ,u max = {pi}
        ,u samples = 23
        ,transformation = {
            matrix_multiply(
                euler(0,pi/2,(2*pi/24)*token.get_macro("angle")/36)
                ,euler(pi/2,pi/3,pi/6)
            )
        }
        ,x = {
            sphere(
                tau *
                token.get_macro("longitude") /
                36
                ,u
            )[1][1]
        }
        ,y = {
            sphere(
                tau *
                token.get_macro("longitude") /
                36
                ,u
            )[1][2]
        }
        ,z = {
            sphere(
                tau *
                token.get_macro("longitude") /
                36
                ,u
            )[1][3]
        }
    ]
}
\foreach \latitude in {1,...,18}{
    \let\latitude\latitude
    \appendcurve[
        u min = 0
        ,u max = {2*pi}
        ,u samples = 4*45
```

```
            ,transformation = {euler(pi/2,pi/3,pi/6)}
            ,x = {
                stereographic_projection(
                    matrix_multiply(
                        sphere(u,tau*token.get_macro("latitude")/36)
                        ,euler(0,pi/2,(tau/24)*token.get_macro("angle")/36)
                    )
                )[1][1]
            }
            ,y = {
                stereographic_projection(
                    matrix_multiply(
                        sphere(u,tau*token.get_macro("latitude")/36)
                        ,euler(0,pi/2,(tau/24)*token.get_macro("angle")/36)
                    )
                )[1][2]
            }
            ,z = 0*u
        ]
        \appendcurve[
            u min = 0
            ,u max = {2*pi}
            ,u samples = 45
            ,transformation = {
                matrix_multiply(
                    euler(0,pi/2,(2*pi/24)*token.get_macro("angle")/36)
                    ,euler(pi/2,pi/3,pi/6)
                )
            }
            ,x = {sphere(u,2*pi*token.get_macro("latitude")/36)[1][1]}
            ,y = {sphere(u,2*pi*token.get_macro("latitude")/36)[1][2]}
            ,z = {sphere(u,2*pi*token.get_macro("latitude")/36)[1][3]}
        ]
    }
    \rendersegments
\end{tikzpicture}
```

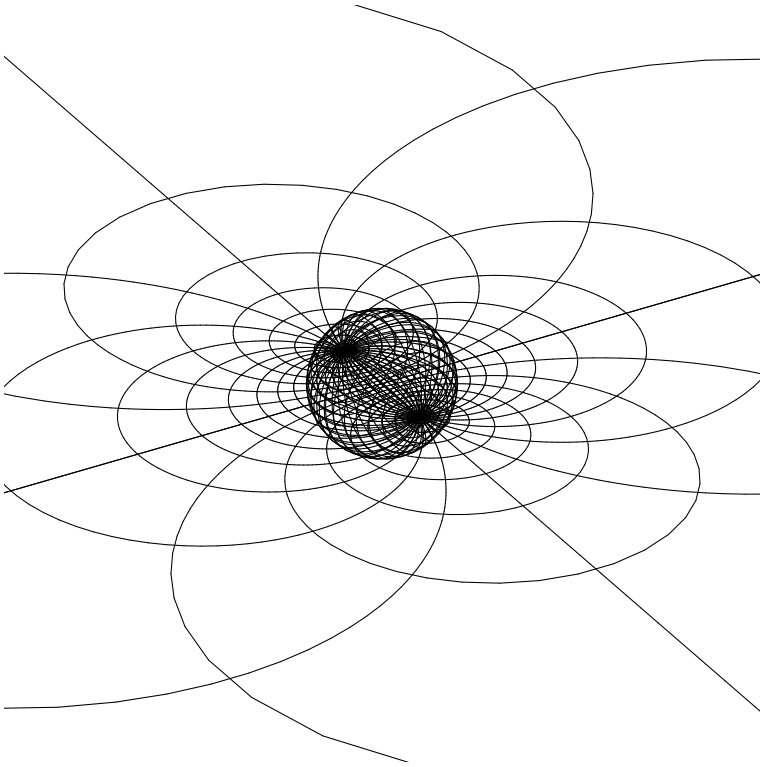

Figure 3.1.2:
Automatic division by zero handling.

Chapter 4

Clipped Subspaces

4.1 Clipping individual planes.